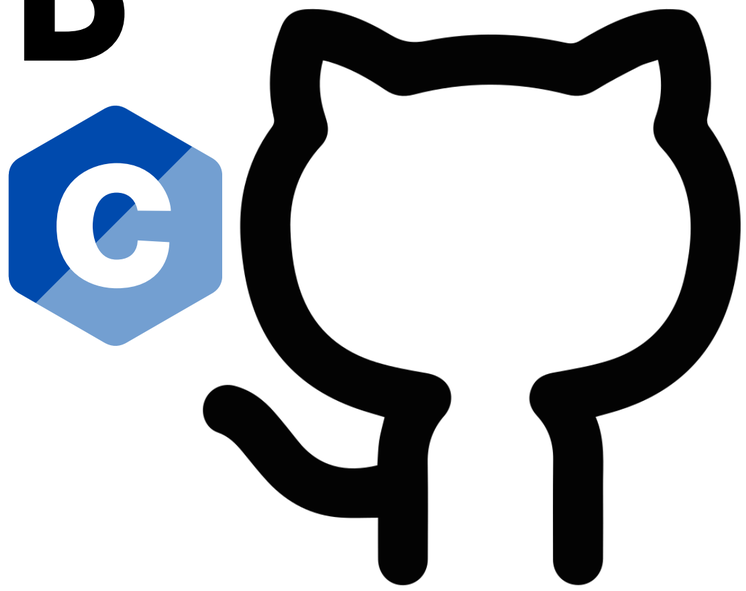
# PROJETO EM C, GIT/GITHUB



Universidade Federal de Viçosa - Campus Florestal
Professora: Thais Regina de Moura Braga Silva
Monitoras: Beatriz Queiroz, Luiza Rosa e Pâmela Lúcia

# AULAS DO LAB'S E TP'S

TP's em duplas

- TP1: Liberar 03/09 - Entrega 24/09 - Entrevistas 29/09 e 01/10
- TP2: Liberar 20/10 - Entrega 29/10 - Entrevistas 03/11 e 05/11
- TP3: Liberar 12/11 - Entrega 24/11 - Entrevistas 01/12 e 03/12

# QUEM SÃO VOCÊS?

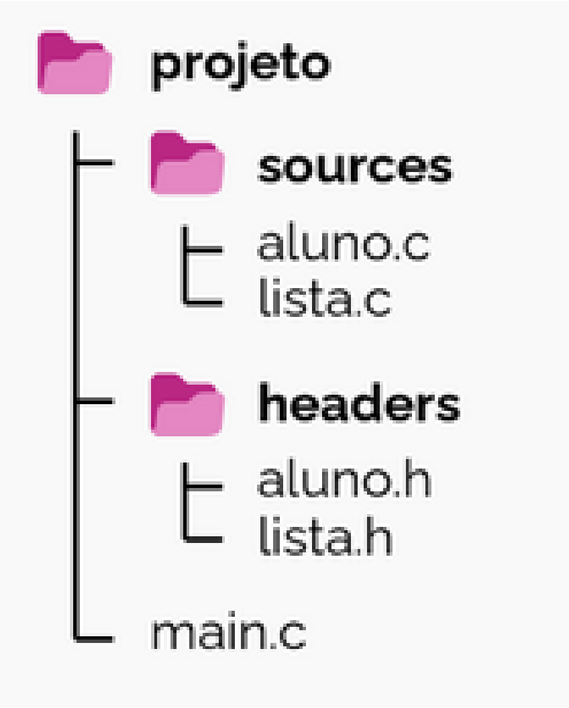
Vamos nos conhecer um pouco: se apresente dizendo seu nome, a cidade de onde veio, o que mais gosta de fazer, e porque escolheu computação!

# REVISÃO DE C

# PROJETO EM C

Em projetos grandes na linguagem C é necessário que sejam criados vários arquivos, de forma a encapsular funções específicas de cada parte do projeto. Para isso são utilizados de dois tipos de arquivo:

- **Arquivos .h**, chamados de cabeçalho, isso porque, eles contêm declarações de funções, variáveis globais, tipos de dados e outras construções que serão usadas por outros arquivos .c.
- **Arquivos .c**, chamados de código fonte, isso porque, eles contêm o código executável propriamente dito, a implementação das funções e algoritmos.



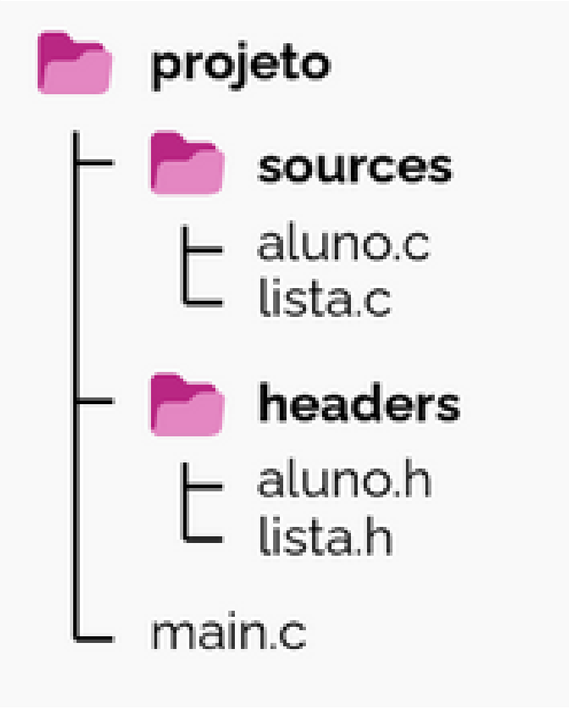

Um exemplo de como utilizar desses arquivos é, por exemplo, a implementação de uma lista de alunos, em que teríamos 5 arquivos principais: o cabeçalho de aluno e da lista, e o código fonte de aluno, da lista, e o main, que realiza a chamada das funções dos mesmos.

# PROJETO EM C

Além de criar os arquivos, é importante entender o funcionamento deles. Como os arquivos .c são o código-fonte de um dado cabeçalho, é necessário que as informações do cabeçalho sejam importadas no mesmo. Assim, por exemplo, no arquivo "aluno.c", teríamos um #include "aluno.h" nele.

É importante ressaltar que, no arquivo "aluno.h" teremos definições como estrutura utilizada para definir um aluno e o cabeçalho das funções, por exemplo, de inserção de um aluno, como inserealuno(Aluno *aluno, char* nome, int matricula);. O código em si dessa função estará no arquivo aluno.c.

Por fim, pensando no main.c, que irá utilizar as funções definidas em cada um dos arquivos mencionados anteriormente, devemos realizar a importação dos arquivos de cabeçalho, como nos demais códigos-fonte.



# PROJETO EM C

```c
#ifndef ALUNO_H
#define ALUNO_H

typedef struct {
    int matricula;
    char nome[50];
} Aluno;

Aluno criar_aluno(int matricula, const char* nome);
void imprimir_aluno(Aluno a);

#endif
```

# PROJETO EM C

```c
#include <stdio.h>
#include <string.h>
#include "aluno.h"

Aluno criar_aluno(int matricula, const char* nome) {
    Aluno a;
    a.matricula = matricula;
    strncpy(a.nome, nome, 49);
    a.nome[49] = '\0';
    return a;
}

void imprimir_aluno(Aluno a) {
    printf("Matricula: %d | Nome: %s\n", a.matricula, a.nome);
}
```

# COMO EXECUTAR VÁRIOS ARQUIVOS EM C

Para executar um projeto completo em C, contendo vários arquivos .c e .h usamos o Makefile. Makefiles são usados para determinar quais partes de um programa grande precisam ser compiladas.

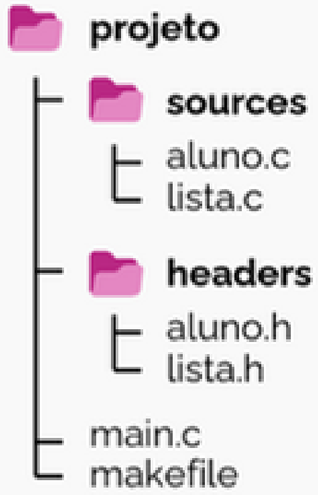
Para utilizar o makefile, é necessário realizar a instalação de acordo com a distribuição do seu sistema operacional.

Para o sistema operacional Windows, é necessário instalar o MinGW Package Manager, que possui um tutorial completo com o passo a passo neste link.

Para sistemas operacionais Linux, o recomendado é utilizar o CMake, que pode ser instalado pelo comando descrito abaixo ou similares. Mais informações sobre a instalação podem ser obtidas neste link.

# COMO EXECUTAR VÁRIOS ARQUIVOS EM C

O arquivo makefile possui dois campos importantes: o primeiro é ‘all’ , que indica quais arquivos são dependentes para aquela execução e o comando a ser chamado, neste caso sendo o comando de compilação da linguagem C; o segundo é ‘run’, que especifica o comando que deve ser chamado para executar o programa após a compilação.




```
all: main.c sources/aluno.c sources/lista.c
     gcc main.c -o exec sources/aluno.c sources/lista.c

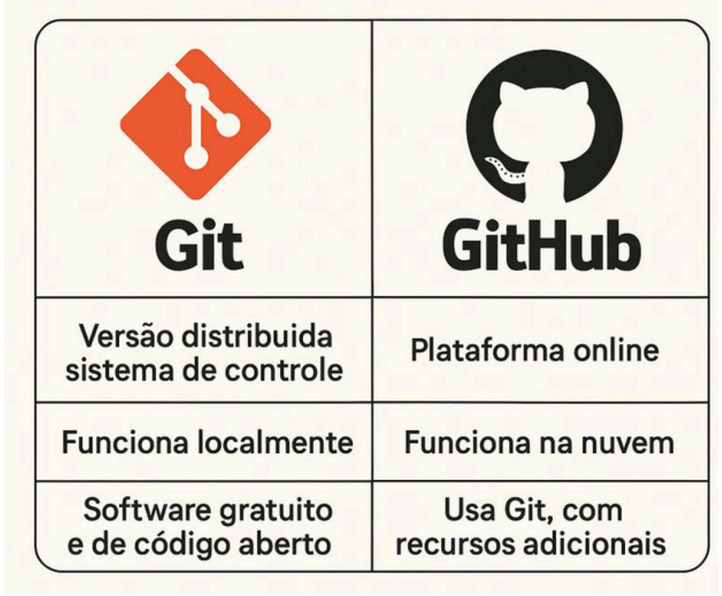
run: ./exec
```

```
> make
> ./exec
```

# O QUE É O GITHUB

O **GitHub** é uma plataforma de desenvolvimento colaborativo que aloja projetos na nuvem utilizando o sistema de controle de versões chamado **Git**. A plataforma ajuda os desenvolvedores a armazenar e administrar o código e faz o registro de mudanças.

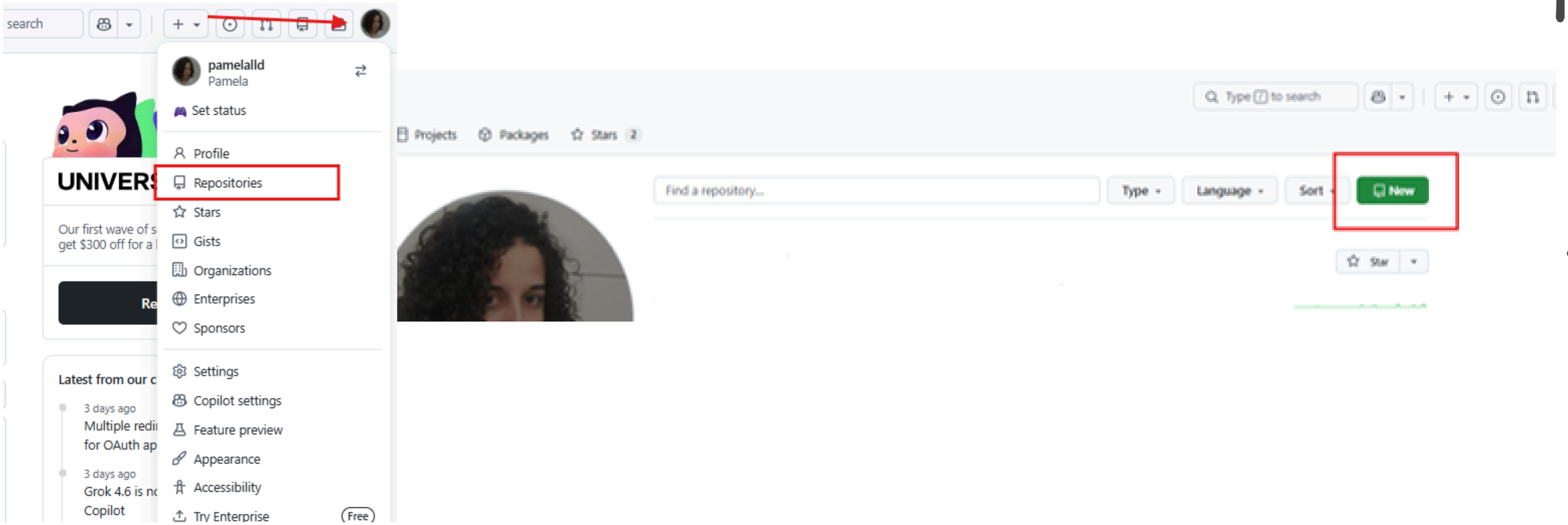
Para começar a utilizar o GitHub, é necessário criar uma conta na plataforma. O que pode ser feito em **https://github.com/**. E como o GitHub utiliza Git, também é necessário instalá-lo no seu computador: **https://git-scm.com/install/**



# CRIANDO UM REPOSITÓRIO

Após criar uma conta no GitHub e a acessá-la, será possível criar repositórios para os seus projetos.

Para acessar os repositórios, basta clicar no seu ícone de perfil (canto superior direito) e em seguida na opção “Repositories”. Depois, basta clicar no botão “New” para criar um novo repositório.



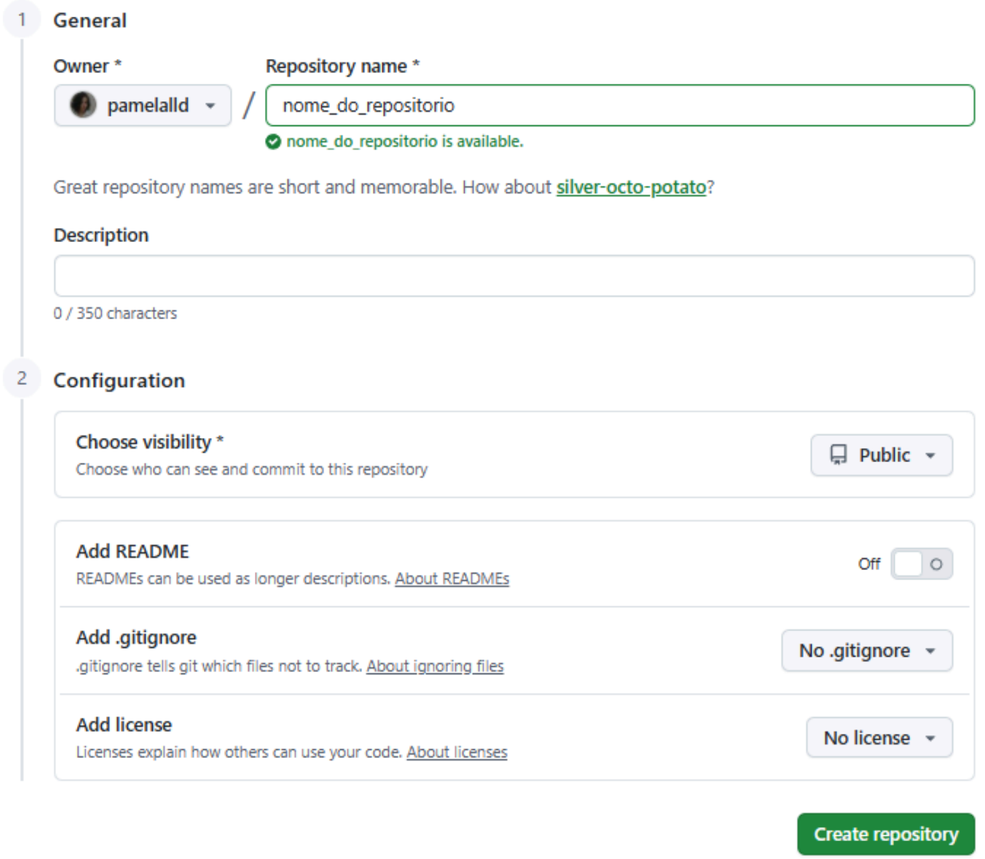
# CRIANDO UM REPOSITÓRIO

Em seguida, você deverá informar um nome para o repositório e opcionalmente uma descrição.

Visibilidade

**Público:** qualquer pessoa poderá visualizar o seu repositório.

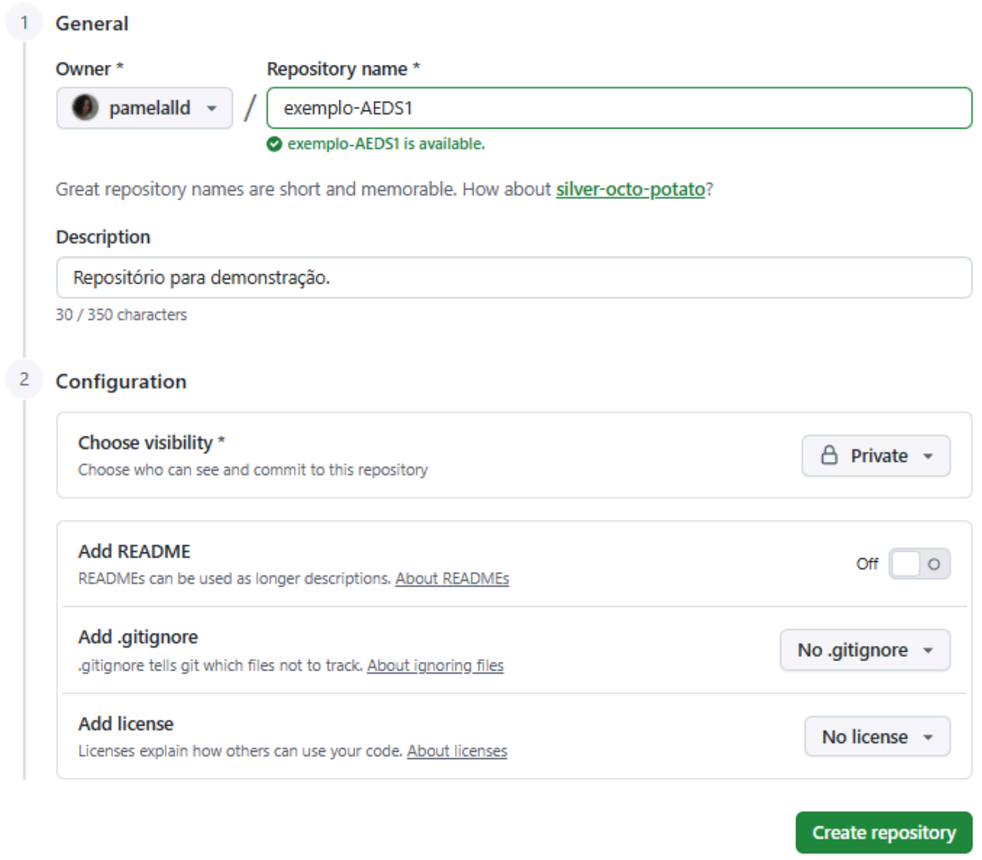
**Privado:** apenas pessoas com permissão poderão acessar o repositório.



# CRIANDO UM REPOSITÓRIO

**README**: é um arquivo que pode ser utilizado para descrever o projeto e colocar instruções de uso. A sua presença é geralmente recomendada.

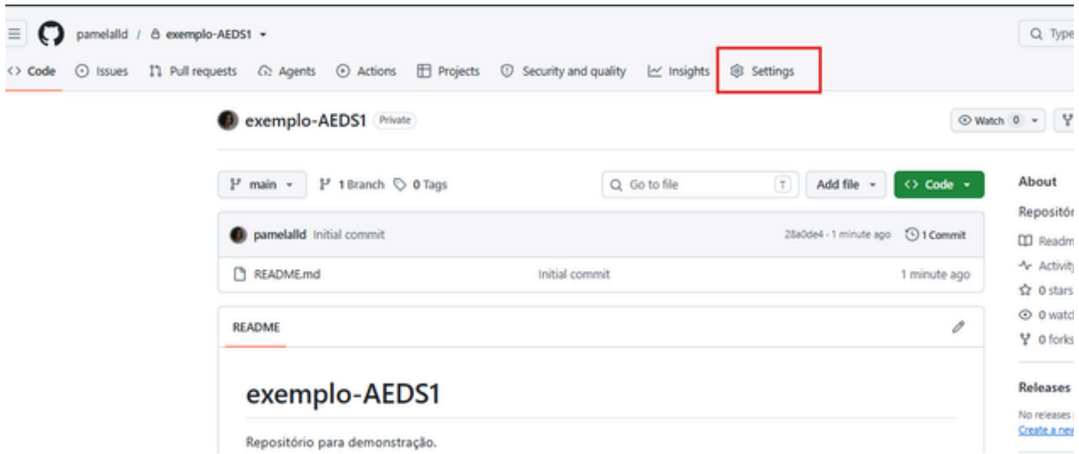
**.gitignore:** é usado para descrever quais pastas ou arquivos devem ser ignorados pelo Git.

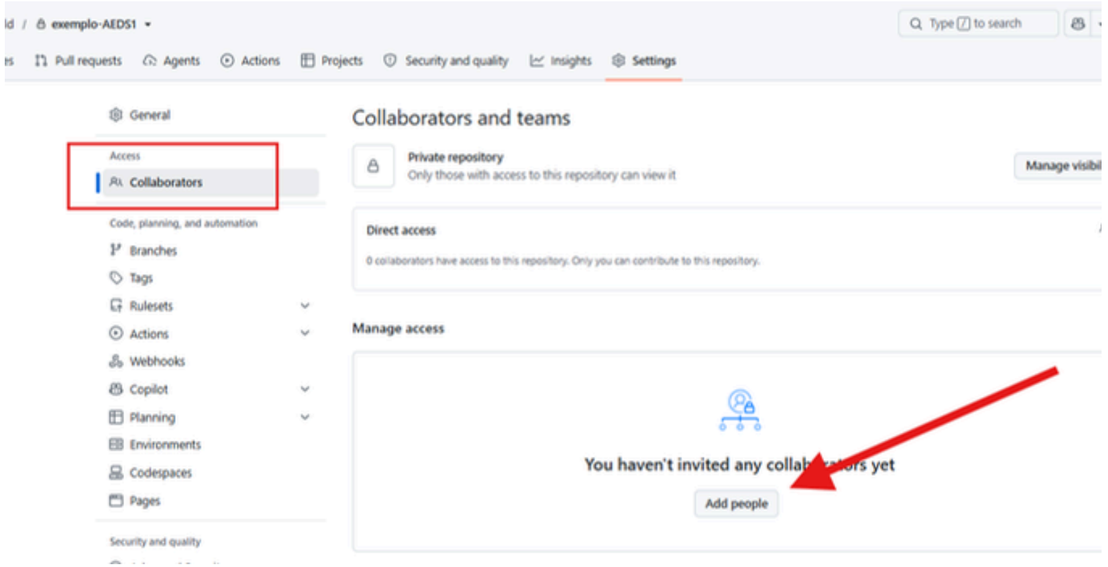


# ADICIONANDO COLABORADORES

Uma vez que o repositório foi criado, é possível adicionar colaboradores ao projeto, que serão as pessoas com permissão para fazer mudanças no conteúdo do repositório.

Para fazer isso, acessem o repositório criado e selecionem a opção “Settings”. Em seguida, acessem a aba “Collaborators”, depois cliquem em “Add people” e informem o nome de usuário da pessoa que desejam adicionar como colaborador do projeto.



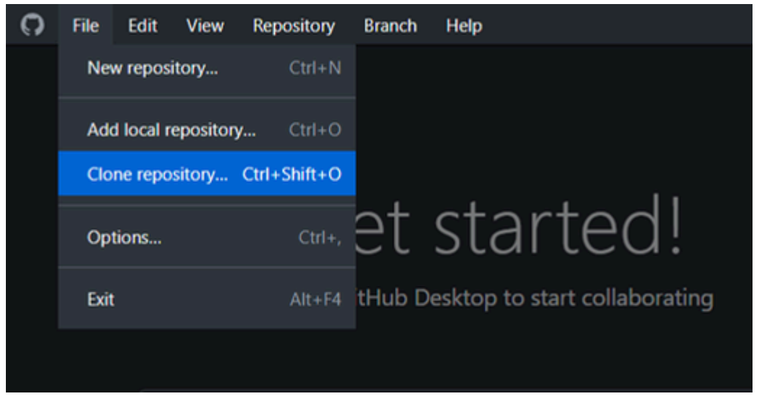

Obs: o colaborador deverá aceitar o convite através do email cadastrado.

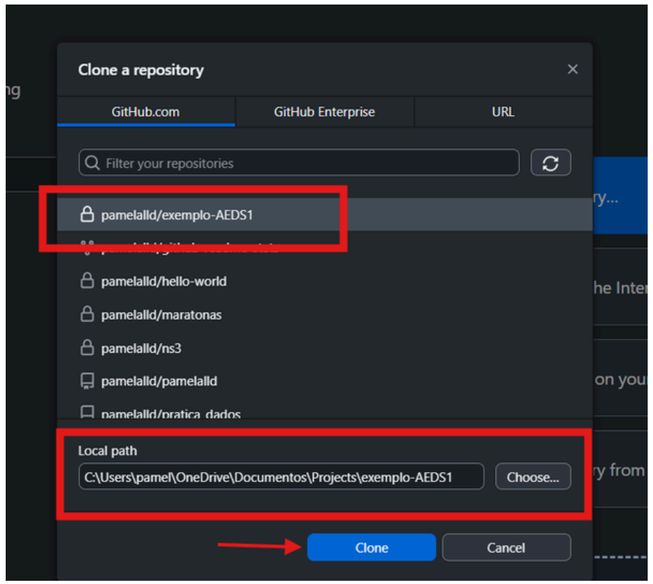
# GITHUB DESKTOP

O GitHub Desktop é um programa que fornece uma interface intuitiva para trabalhar com Git e GitHub, vocês podem instalá-lo **aqui**.

Uma vez instalado, vocês devem fazer o login com a sua conta do GitHub para trabalhar com os seus repositórios.

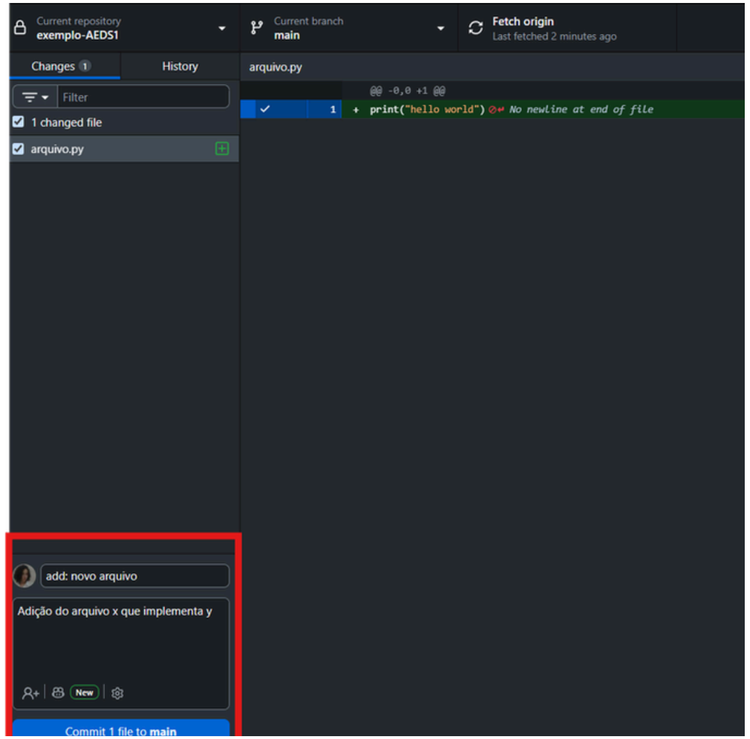
Agora, clique na opção “File” e em seguida em “Clone repository”, isso vai permitir que você traga os repositórios remotos para o seu computador. Por fim, selecione o repositório que deseja e também o diretório do computador onde pretende salvá-lo e clique no botão “Clone”.

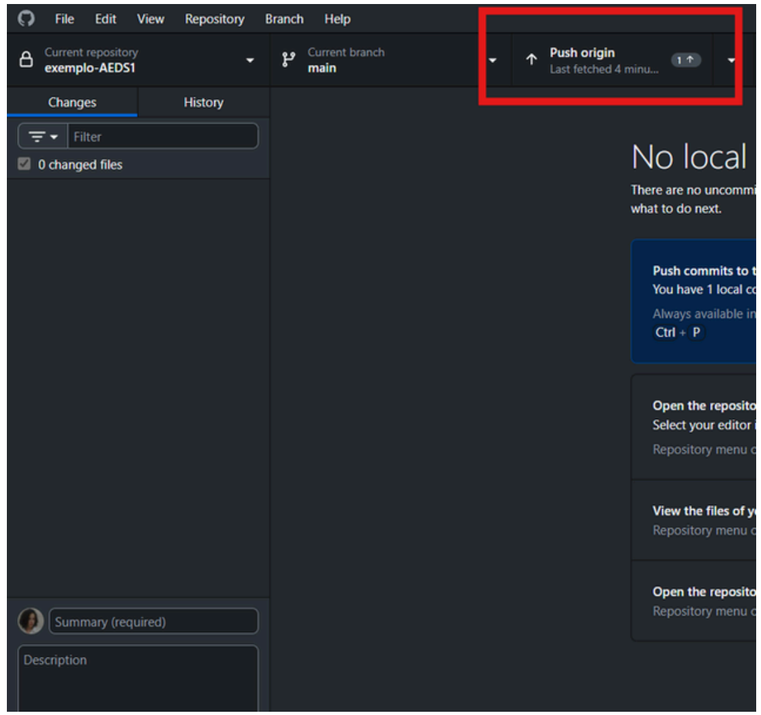


# GITHUB DESKTOP

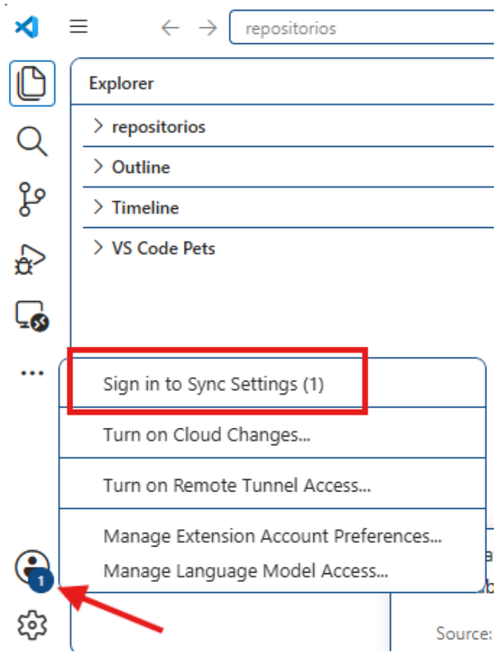
Sempre que forem feitas alterações no projeto, elas serão exibidas no GitHub Desktop. Para enviar as mudanças ao repositório remoto, você deve adicionar um comentário informativo e clicar no botão “Commit”.
Em seguida, você deve confirmar o envio das alterações clicando na opção “Push Origin”.



# GITHUB + VSCODE

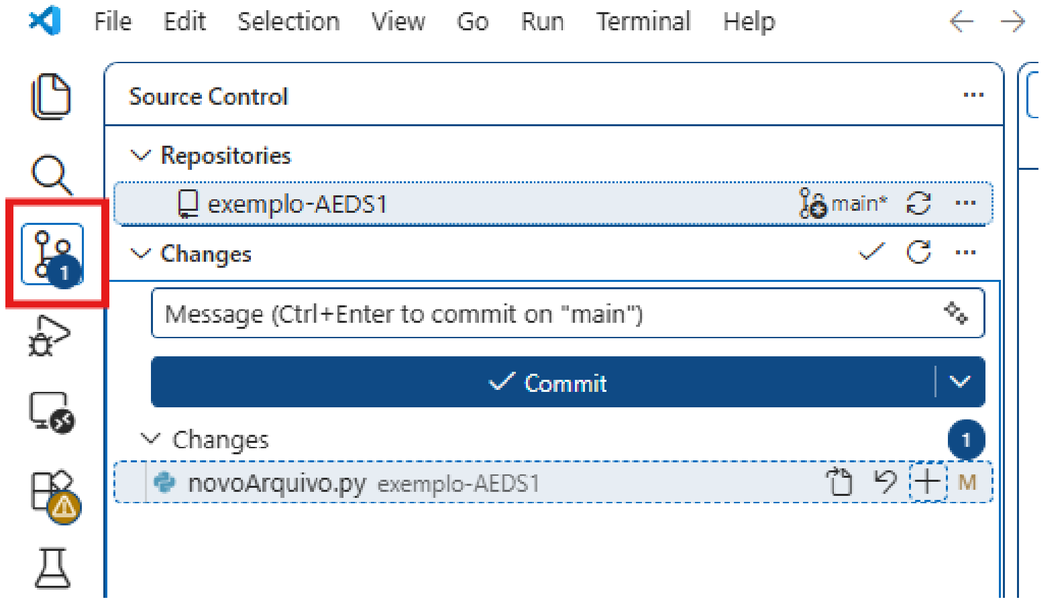
Para fazer o versionamento diretamente pelo VSCode, acesse o ícone no canto inferior esquerdo e a opção “Sign in”, em seguida você deverá conceder permissão para que o VSCode tenha acesso à sua conta do GitHub. Para clonar um repositório, utilize no próprio terminal do VSCode, no diretório desejado, o comando **git clone linkdorepositorio**

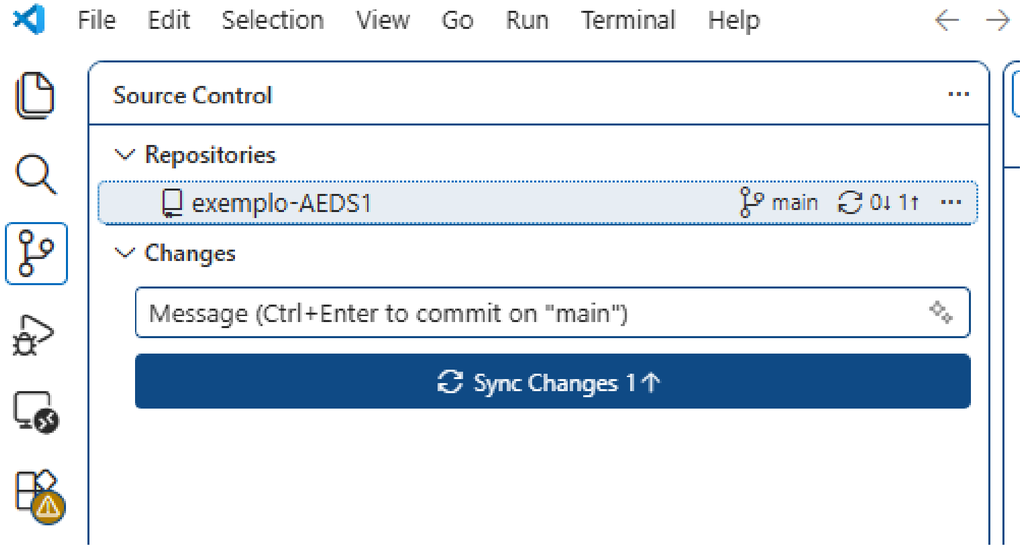





# GITHUB + VSCODE

Para enviar as suas mudanças ao repositório remoto, acesse o terceiro ícone do painel esquerdo. Na seção “Changes” serão listados todos os arquivos alterados, verifique cada um e clique no botão “+” para adicionar essa mudança ao commit. Em seguida, escreva uma mensagem descrevendo a mudança e clique em “Commit”. Depois, clique em “Sync Changes” para sincronizar com o repositório remoto.



# COMANDOS MAIS USADOS

Todas as ações também podem ser realizadas através de comandos Git. A seguir, estão detalhados alguns dos comandos básicos para utilização do Git e do GitHub.

**git clone:** Utilizado para baixar uma cópia de um repositório remoto para o computador.

git clone URL_DO_REPOSITORIO

Exemplo:
*git clone https://github.com/usuario/projeto.git*

**git pull:** Utilizado para baixar as alterações mais recentes do repositório remoto e atualizar a versão do projeto que está no computador.

*git pull*

# COMANDOS MAIS USADOS

**git add:** Utilizado para adicionar as alterações ao staging, preparando-as para serem incluídas no próximo commit.

Para adicionar um arquivo específico:
*git add nome-do-arquivo*

Para adicionar todos os arquivos modificados:
*git add .*

**git commit:** Utilizado para registrar as alterações.

Exemplo:
*git commit -m "Adiciona função de soma"*

**git push:** Utilizado para enviar os commits realizados localmente para o repositório remoto.

*git push*

# CONTEÚDOS AUXILIARES

Documentação GitHub: https://docs.github.com/pt

Lista de comandos: https://gist.github.com/leocomelli/2545add34e4fec21ec16

Trabalhando com Git e GitHub no VSCode: https://www.youtube.com/watch?v=7cNP3AE49Bg

# OBRIGADO!